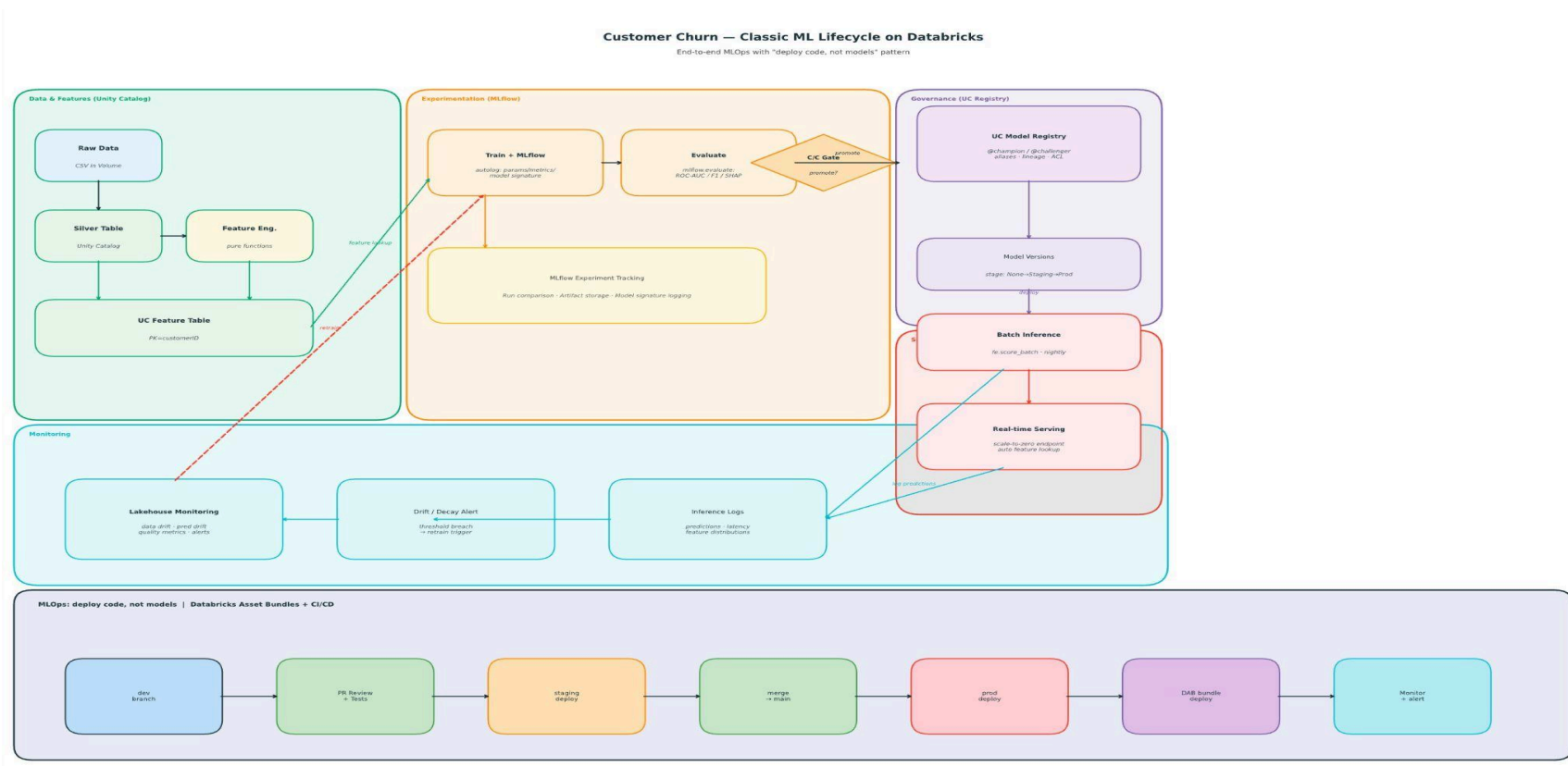


Databricks MLflow — Classic ML Lifecycle

Dev to Prod - A practical reference: Customer Churn prediction, end to end



Databricks MLflow — Classic ML Lifecycle	1
Dev to Prod - A practical reference: Customer Churn prediction, end to end	1
0. The mental model (read this first)	4
1. Environment setup	5
The three-environment layout	5
Minimal project skeleton	5
✓ Best practices	6
2. Data & feature engineering with Unity Catalog	7
2.1 Land raw data as a governed table	7
2.2 Build features as pure functions (testable!)	7
2.3 Write to a Feature Engineering (UC) table	8
✓ Best practices	8
3. Experiment tracking with MLflow	9
3.1 Autolog does 80% for free	9
3.2 Train inside a run, using the feature table	9
✓ Best practices	10
4. Model evaluation	11
4.1 Use mlflow.evaluate for a standard, comparable report	11
Which metric for churn?	11
✓ Best practices	12
5. Model Registry in Unity Catalog	13
5.1 Register to Unity Catalog (not the old workspace registry)	13
5.2 Aliases, not stages	13
✓ Best practices	14
6. Model serving	15
6.1 Batch inference (most churn use cases)	15
6.2 Real-time serving (low-latency decisions)	15
Batch vs. real-time — how to choose	16
✓ Best practices	16

7. Monitoring & retraining.....	17
What to watch.....	17
The retraining loop.....	18
✅ Best practices.....	18
8. MLOps automation: "deploy code, not models".....	19
Why deploy code, not models?.....	19
8.1 Databricks Asset Bundles (DABs) — infrastructure as code.....	19
8.2 The champion/challenger gate (evaluate.py).....	21
8.3 CI/CD (GitHub Actions example).....	22
✅ Best practices.....	22
9. One-page lifecycle checklist.....	24
10. Common pitfalls (and the fix).....	25

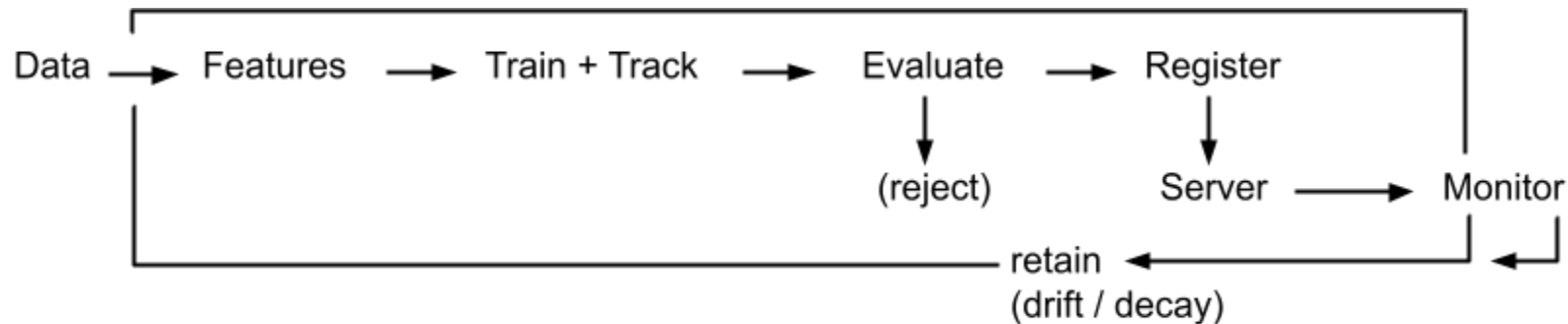
How to read this doc. Sections follow the ML lifecycle in order:

Setup → Data & Features → Experiment Tracking → Evaluation → Model Registry → Serving → Monitoring → MLOps Automation.

Each section has a short **Why it matters** callout, the **key code**, and a ✅ **Best practices** list. Beginners can read top-to-bottom; experienced readers can jump to any section as a standalone reference

0. The mental model (read this first)

Machine learning in production is not "train a model." It is a **loop** that keeps a model useful as the world changes:



Two ideas make everything else click:

1. **MLflow is the system of record.** Every experiment, metric, artifact, and model version is logged so any result is reproducible and auditable. If it isn't in MLflow, it didn't happen.
2. **Unity Catalog (UC) is the governance layer.** Data, features, and models all live as governed UC assets (`catalog.schema.object`) with lineage, permissions, and discoverability. One security model for everything.

The golden rule of Databricks MLOps: _deploy code, not models._ You do not train a model in a notebook and copy the `.pkl` to production. You promote the **training pipeline code** through dev → staging → prod, and each environment runs that code against its own data and registers its own model. This is covered fully in [#8](#).

1. Environment setup

Why it matters: Reproducibility starts with a pinned, shared environment. "Works on my cluster" is the ML equivalent of "works on my machine."

The three-environment layout

Environment	Purpose	Who writes here	Data
dev	Experimentation, feature/model iteration	Data scientists	Sample or full dev data
staging	Integration tests, CI validation of the pipeline	CI/CD only	Prod-like data
prod	Scheduled training + serving	CI/CD only	Real production data

Prefer **three separate workspaces** (or at minimum three UC catalogs: dev, staging, prod) so permissions and blast radius are cleanly separated.

Minimal project skeleton

```
churn-mlops/
├─ databricks.yml      # Asset Bundle: defines jobs, targets (dev/staging/prod)
├─ requirements.txt    # Pinned deps for reproducibility
├─ src/
│   ├─ features.py     # Feature engineering (pure, testable functions)
│   ├─ train.py        # Training + MLflow logging entrypoint
│   └─ evaluate.py     # Champion/Challenger gate
├─ tests/
│   └─ test_features.py # Unit tests on feature logic
└─ resources/
```

```
└─ jobs.yml          # Job definitions referenced by the bundle
```

Why a repo, not just notebooks? Notebooks are great for exploration but hard to test, review, and version. Put reusable logic in `src/*.py`, import it into thin notebooks or job tasks, and unit-test it. This is the single biggest step from "notebook data science" to "engineering."

✓ Best practices

- Pin the **runtime**: use a Databricks Runtime **ML** version (MLflow, feature-engineering libs preinstalled) and pin it in the bundle.
- Pin Python deps in `requirements.txt`; MLflow also captures them automatically per-run (see §3).
- Store code in **Git** (Databricks Repos / Git folders). Never let prod run from an ad-hoc notebook.

2. Data & feature engineering with Unity Catalog

Why it matters: Most model failures are data failures. Governed, versioned, reusable features prevent the two classic bugs: **training/serving skew** (features computed differently at train vs. inference time) and **data leakage** (using information at training time that wouldn't exist at prediction time).

2.1 Land raw data as a governed table

```
# Read raw churn data and write a managed UC table (Bronze → Silver pattern)
raw = spark.read.csv("/Volumes/dev/churn/landing/telco.csv", header=True, inferSchema=True)
```

```
(raw
 .write
 .mode("overwrite")
 .saveAsTable("dev.churn.customers_silver")) # catalog.schema.table
```

2.2 Build features as pure functions (testable!)

```
# src/features.py
from pyspark.sql import DataFrame, functions as F

def compute_customer_features(df: DataFrame) -> DataFrame:
    """Pure transformation: same input -> same output. Easy to unit test."""
    return (df
        .withColumn("tenure_years", F.col("tenure") / 12.0)
        .withColumn("avg_monthly_spend", F.col("TotalCharges") / F.greatest(F.col("tenure"), F.lit(1)))
        .withColumn("is_month_to_month", (F.col("Contract") == "Month-to-month").cast("int"))
        .select("customerID", "tenure_years", "avg_monthly_spend", "is_month_to_month", "MonthlyCharges")
    )
```

2.3 Write to a Feature Engineering (UC) table

Why a feature table (vs. a plain table)? A UC feature table has a **primary key** and is understood by the Feature Engineering client. When you train with it, MLflow records *which features the model consumes*. At serving time Databricks can **automatically look up those features by key** — so your online request only needs `customerID`, and the platform fetches the rest. This is what kills training/serving skew.

```
from databricks.feature_engineering import FeatureEngineeringClient

fe = FeatureEngineeringClient()
features_df = compute_customer_features(spark.table("dev.churn.customers_silver"))

fe.create_table(
    name="dev.churn.customer_features",
    primary_keys=["customerID"],
    df=features_df,
    description="Per-customer churn features. PK=customerID."
)
```

✓ Best practices

- **Point-in-time correctness:** for time-dependent features, join labels to features *as of the prediction time*, not "now." Use feature table time-series keys or point-in-time joins. This prevents leakage.
- Keep feature logic in **pure functions** in `src/` and unit-test them (`tests/test_features.py`).
- One feature table per **entity** (customer, account) with a clear PK. Reuse across models/teams.
- Let UC track **lineage** — you'll be able to see table → feature → model → serving endpoint automatically.

3. Experiment tracking with MLflow

Why it matters: You will train dozens of models. Tracking makes results **comparable, reproducible, and shareable** — no more "which run had 0.87 AUC?" screenshots.

3.1 Autolog does 80% for free

```
import mlflow

mlflow.set_experiment("/Users/you@co.com/churn") # or a shared workspace path
mlflow.sklearn.autolog() # logs params, metrics, model, signature, requirements automatically
```

`autolog()` captures hyperparameters, training metrics, the model artifact, an **input signature** (schema), and the **environment** (pip deps) — all without extra code.

3.2 Train inside a run, using the feature table

Using `fe.create_training_set` (not a plain `DataFrame`) is what records feature lineage into the model, enabling automatic lookups at serving time.

```
from databricks.feature_engineering import FeatureEngineeringClient, FeatureLookup
from sklearn.ensemble import GradientBoostingClassifier
import mlflow
```

```
fe = FeatureEngineeringClient()
```

```
# Labels: customerID + churn outcome
labels = spark.table("dev.churn.customers_silver").select("customerID", "Churn")
```

```
training_set = fe.create_training_set(
    df=labels,
```

```

    feature_lookups=[FeatureLookup(table_name="dev.churn.customer_features", lookup_key="customerID")],
    label="Churn",
    exclude_columns=["customerID"],
)
train_pdf = training_set.load_df().toPandas()
X, y = train_pdf.drop("Churn", axis=1), (train_pdf["Churn"] == "Yes").astype(int)

with mlflow.start_run(run_name="gbt-baseline") as run:
    model = GradientBoostingClassifier(n_estimators=200, max_depth=3)
    model.fit(X, y)

# log_model via the FE client bakes feature metadata into the model
fe.log_model(
    model=model,
    artifact_path="model",
    flavor=mlflow.sklearn,
    training_set=training_set,          # <-- records feature lineage
    registered_model_name="dev.churn.churn_model", # optional: register immediately
)

```

✓ Best practices

- **Name your runs** and set tags (`git_sha`, `data_version`, `author`) so runs are searchable and reproducible.
- Log a **model signature** (autolog does this) — it prevents malformed inputs at serving time.
- Use **nested runs** for hyperparameter sweeps (parent run = the sweep, child runs = each trial).
- Prefer `fe.log_model` over `mlflow.log_model` when you trained from a feature table — it enables serving-time lookups.

4. Model evaluation

Why it matters: A single accuracy number lies, especially on imbalanced data (churners are usually the minority). Evaluate with the metrics that match the **business decision**, and always compare against a baseline.

4.1 Use `mlflow.evaluate` for a standard, comparable report

```
import mlflow

with mlflow.start_run(run_name="gbt-baseline-eval"):
    result = mlflow.evaluate(
        model=f"runs:{run.info.run_id}/model",
        data=eval_pdf,          # holdout set incl. label
        targets="Churn",
        model_type="classifier",
        evaluators="default",
    )
    print(result.metrics)  # accuracy, f1, roc_auc, precision/recall, confusion matrix, SHAP...
```

`mlflow.evaluate` logs metrics **and** explainability plots (SHAP), a confusion matrix, and ROC/PR curves as run artifacts — so evaluation is reproducible and reviewable, not a one-off notebook cell.

Which metric for churn?

Metric	Use when	Churn note
ROC-AUC	Ranking quality, threshold-independent	Good overall model-quality signal
PR-AUC / F1	Positive class is rare	Often the right primary metric for churn

Recall @ k	You can only act on top-k customers	Matches a "retention team calls 500/week" budget
Calibration	You use probabilities in \$ decisions	Needed if churn prob feeds an expected-value calc

Best practices

- Fix a **holdout / time-based split** — never evaluate on training rows.
- Choose **one primary metric** tied to the business action, before you start tuning (prevents metric-shopping).
- Check **subgroup performance** (by contract type, tenure band) to catch fairness/robustness gaps.
- Store the eval dataset version so results are reproducible.

5. Model Registry in Unity Catalog

Why it matters: The registry is the **hand-off point** between experimentation and production. It gives every model version an identity, lineage, permissions, and a lifecycle — the thing serving and CI/CD point at.

5.1 Register to Unity Catalog (not the old workspace registry)

```
import mlflow
mlflow.set_registry_uri("databricks-uc")    # models live at catalog.schema.name
# (fe.log_model above already registered dev.churn.churn_model)
```

5.2 Aliases, not stages

Modern pattern: UC model registry uses **aliases** (mutable, human-meaningful pointers) instead of the deprecated **Staging/Production** stages. Typical aliases: **@champion** (currently serving), **@challenger** (candidate under test).

```
from mlflow.tracking import MlflowClient
client = MlflowClient()
```

```
# Promote a specific version to challenger
client.set_registered_model_alias("dev.churn.churn_model", "challenger", version=3)
```

```
# Load by alias in downstream code (serving, batch scoring)
champion = mlflow.pyfunc.load_model("models:/dev.churn.churn_model@champion")
```

Why aliases beat stages: they're arbitrary and meaningful (**@champion**, **@shadow**, **@eu_region**), they decouple "which version is live" from code, and promotion is a single atomic pointer move that you can automate and audit.

✓ Best practices

- Use `@champion`/`@challenger`` aliases; drive promotion from CI, not by hand in the UI.
- Attach **tags** and a **description** to each version (metrics, git sha, approver).
- Govern with UC **permissions**: only the CI service principal can set the `@champion` alias in prod.
- One registered model name per use case; version numbers are automatic and immutable.

6. Model serving

Why it matters: A model creates value only when predictions reach a decision. Databricks supports the two patterns you'll actually use:

6.1 Batch inference (most churn use cases)

Best when you score a whole population on a schedule (e.g., nightly churn scores for the retention team).

```
from databricks.feature_engineering import FeatureEngineeringClient
fe = FeatureEngineeringClient()

# Only need the keys – features are looked up automatically from the feature table
scoring_keys = spark.table("prod.churn.active_customers").select("customerID")

preds = fe.score_batch(
    model_uri="models:/prod.churn.churn_model@champion",
    df=scoring_keys,
)
preds.write.mode("overwrite").saveAsTable("prod.churn.daily_scores")
```

6.2 Real-time serving (low-latency decisions)

Best when a prediction is needed in-app (e.g., churn risk shown when an agent opens a customer record).

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.serving import EndpointCoreConfigInput, ServedEntityInput

w = WorkspaceClient()
w.serving_endpoints.create(
    name="churn-endpoint",
```

```

config=EndpointCoreConfigInput(
    served_entities=[ServedEntityInput(
        entity_name="prod.churn.churn_model",
        entity_version="",          # or pin; alias-based serving supported
        scale_to_zero_enabled=True # cost control: scales down when idle
    )]
),
)

```

With **automatic feature lookup**, the online request sends just `{"customerID": "1234"}`; the endpoint fetches features from an **online store** and returns a score. No feature logic duplicated in the app = no skew.

Batch vs. real-time — how to choose

	Batch	Real-time
Latency	Minutes–hours	Milliseconds
Cost	Cheapest (job compute)	Endpoint (use scale-to-zero)
Use when	Score everyone on a schedule	Score one entity on demand
Churn example	Nightly retention list	Risk badge in the CRM UI

✓ Best practices

- Default to **batch** unless you truly need sub-second latency — it's simpler and cheaper.
- Enable **scale-to-zero** on real-time endpoints to avoid paying for idle.
- Serve by **alias** (@champion) so promotion doesn't require redeploying the endpoint.
- Turn on **inference tables** (auto-logging of requests/responses) — you'll need them for \$7.

7. Monitoring & retraining

Why it matters: Models decay. Customer behavior, pricing, and competitors change, so a model trained last quarter silently gets worse. Monitoring turns "silently worse" into "alerted and retrained."

What to watch

Signal	Question it answers	Tool
Data drift	Are inputs different from training data?	Lakehouse Monitoring (data profile)
Prediction drift	Is the score distribution shifting?	Lakehouse Monitoring
Model quality	Are predictions still accurate?	Join predictions to actual outcomes
Ops health	Latency, error rate, throughput	Endpoint metrics / inference tables

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.catalog import MonitorInferenceLog

w = WorkspaceClient()
w.quality_monitors.create(
    table_name="prod.churn.daily_scores",
    inference_log=MonitorInferenceLog(
        timestamp_col="scored_at",
        prediction_col="prediction",
        model_id_col="model_version",
        label_col="actual_churn",          # joined later when outcomes are known
        problem_type="PROBLEM_TYPE_CLASSIFICATION",
        granularities=["1 day"],
    ),
    output_schema_name="prod.churn",
```

)

Lakehouse Monitoring auto-creates **profile** and **drift** metric tables plus a dashboard. Set **alerts** (Databricks SQL alerts) on drift thresholds or an AUC drop.

The retraining loop

- **Trigger:** scheduled (e.g., weekly), or event-driven (drift alert fires).
- **Action:** the *same* training pipeline (§8) reruns → produces a @challenger → the eval gate compares it to @champion → promotes only if better.

✓ Best practices

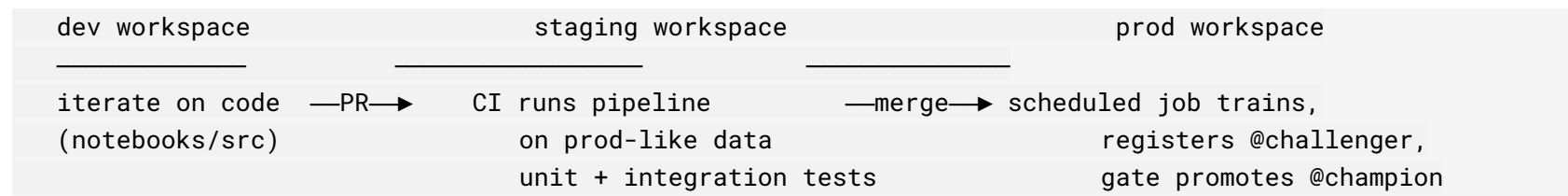
- Ground-truth labels arrive **late** (a customer's churn is known weeks later) — build a delayed join to compute true quality.
- Alert on **business impact**, not just statistics (e.g., "retention list precision < 0.4").
- Automate retraining as a **gated** pipeline; never auto-promote without the challenger beating the champion.

8. MLOps automation: "deploy code, not models"

Why it matters: This is the difference between a demo and a production system. Instead of moving fragile model files between environments, you move **reviewed, versioned code**, and each environment produces its own model with full lineage.

Why deploy code, not models?

- **Reproducibility:** prod trains from code in Git at a known commit — you can always rebuild the exact model.
- **Governance:** prod data never leaves prod; the dev model never touches prod. Only *code* crosses the boundary, via PR review.
- **Simplicity:** one artifact type to promote (code), tested by CI, instead of syncing model binaries + environments + feature definitions across workspaces.



8.1 Databricks Asset Bundles (DABs) — infrastructure as code

A bundle (`databricks.yml`) declares your jobs, their code, and per-environment **targets**. One file describes the whole deployment; `databricks bundle deploy -t prod` provisions it identically everywhere.

```
# databricks.yml
bundle:
  name: churn-mlops
```

```
variables:
  catalog:
    description: UC catalog for this environment
```

```
targets:
  dev:
    mode: development          # prefixes resources, pauses schedules
    default: true
    variables: {catalog: dev}
    workspace: {host: https://dev.cloud.databricks.com}
  staging:
    variables: {catalog: staging}
    workspace: {host: https://staging.cloud.databricks.com}
  prod:
    mode: production           # enforces prod guardrails
    variables: {catalog: prod}
    workspace: {host: https://prod.cloud.databricks.com}
    run_as: {service_principal_name: churn-ci-sp} # runs as SP, not a person
```

```
resources:
  jobs:
    churn_training:
      name: "[${bundle.target}] churn-training"
      tasks:
        - task_key: train
          notebook_task: {notebook_path: ./src/train.py}
          job_cluster_key: ml
        - task_key: evaluate_and_gate
          depends_on: [{task_key: train}]
          notebook_task: {notebook_path: ./src/evaluate.py}
      schedule:
        quartz_cron_expression: "0 0 8 ? * MON" # weekly retrain
        pause_status: UNPAUSED
```

```

    job_clusters:
      - job_cluster_key: ml
        new_cluster: {spark_version: "15.4.x-cpu-ml-scala2.12", num_workers: 2, node_type_id:
i3.xlarge}

```

The `${bundle.target}` and `${var.catalog}` substitutions mean **the same code** targets `dev.churn.*`, `staging.churn.*`, or `prod.churn.*` with zero edits — the essence of "deploy code, not models."

8.2 The champion/challenger gate (`evaluate.py`)

Promotion is **conditional**, automated, and auditable:

```

# src/evaluate.py (runs as a job task after training)
import mlflow
from mlflow.tracking import MlflowClient

```

```

client = MlflowClient()
MODEL = "prod.churn.churn_model"

```

```

challenger = mlflow.pyfunc.load_model(f"models:{MODEL}@challenger")
new_auc = mlflow.evaluate(model=f"models:{MODEL}@challenger", data=holdout,
                          targets="Churn", model_type="classifier").metrics["roc_auc"]

```

```

try:
    champ_ver = client.get_model_version_by_alias(MODEL, "champion").version
    old_auc = float(client.get_model_version(MODEL, champ_ver).tags["roc_auc"])
except Exception:
    old_auc = 0.0 # no champion yet → first model wins

```

```

if new_auc > old_auc + 0.01: # require a meaningful improvement
    ver = client.get_model_version_by_alias(MODEL, "challenger").version

```

```

    client.set_registered_model_alias(MODEL, "champion", ver)    # promote
    print(f"Promoted v{ver}: {old_auc:.3f} -> {new_auc:.3f}")
else:
    print(f"Kept champion: challenger {new_auc:.3f} did not beat {old_auc:.3f}")

```

8.3 CI/CD (GitHub Actions example)

```

# .github/workflows/deploy.yml
on:
  pull_request: {branches: [main]}    # -> validate + deploy to staging
  push: {branches: [main]}           # -> deploy to prod

jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: databricks/setup-cli@main
      - run: pytest tests/                # unit-test feature logic
      - run: databricks bundle validate -t staging
      - if: github.event_name == 'pull_request'
        run: databricks bundle deploy -t staging    # integration test on prod-like data
      - if: github.ref == 'refs/heads/main'
        run: databricks bundle deploy -t prod      # promote code to prod

```

Best practices

- **Everything as code:** jobs, clusters, schedules, permissions live in the bundle and in Git.
- Prod jobs **run as a service principal**, never a personal identity.

- CI runs **unit tests** (feature/transform logic) + a **pipeline smoke test** in staging before prod.
- Promotion is a **gated, automated** alias move — humans review code in PRs, not models in a UI.
- Keep secrets in **Databricks secret scopes**, never in code.

9. One-page lifecycle checklist

Stage	Do this	The "why" in one line
Setup	Git repo, pinned ML runtime, dev/staging/prod catalogs	Reproducibility & isolation
Data	Land as UC tables; features as pure functions	Testable, governed, no skew
Features	UC feature table with PK; point-in-time joins	Reuse + no leakage
Track	mlflow.autolog(), named runs, tags	Comparable, reproducible runs
Evaluate	mlflow.evaluate, one business-tied metric	Honest, comparable quality
Register	UC registry, @champion/@challenger aliases	Governed hand-off point
Serve	Batch by default; real-time + scale-to-zero if needed	Value reaches the decision
Monitor	Lakehouse Monitoring + alerts + delayed labels	Catch decay before users do
Automate	Asset Bundles + CI/CD + gated promotion	Deploy code, not models

10. Common pitfalls (and the fix)

- **Training/serving skew** → compute features once in a UC feature table; use automatic lookup at serving.
 - **Data leakage** → point-in-time-correct joins; never use post-outcome columns.
 - **"Best" model chosen on a shifting metric** → fix one primary metric before tuning.
 - **Copying `.pkl` to prod** → deploy code; let prod register its own model.
 - **Manual UI promotion** → automate the champion/challenger alias move in CI.
 - **No monitoring** → you'll learn about decay from an angry stakeholder. Add drift + quality monitors on day one.
 - **Personal identity runs prod** → use a service principal; people leave, pipelines shouldn't break.
-

Glossary (fast reference)

- **MLflow** — open-source platform for tracking experiments, packaging models, and managing their lifecycle.
- **Unity Catalog (UC)** — Databricks' unified governance layer for data, features, and models (`catalog.schema.object`).
- **Feature table** — a governed UC table with a primary key, usable for training and automatic serving-time lookups.
- **Alias** — a mutable pointer to a model version (e.g., `@champion`); replaces deprecated stages.
- **Asset Bundle (DAB)** — infrastructure-as-code for Databricks jobs/pipelines/resources across environments.
- **Champion/Challenger** — the live model vs. a candidate; promote only when the challenger wins a gate.
- **Lakehouse Monitoring** — managed drift/quality monitoring that auto-generates metric tables and dashboards.